

CAB230 Assignment 2 - Client Side Application Final Report

Application: CAB230 Rentals API Client

Student name: Huynh Anh Quan Dang Student number: n12051632

Date: 01 May 2026

Introduction

Purpose & Description

This application is a client-side React single-page application for exploring the CAB230 Rentals API. It allows users to search Australian rental properties, filter and sort results, view individual property details, inspect property locations on an interactive map, register and log in, submit property ratings, and review the rentals they have previously rated.

The app is designed as a practical rental comparison tool. The main workspace is the Rental Search page, supported by a persistent navigation bar, detailed property pages, authentication workflows, and a protected Rated Rentals history page. The implementation uses React Router for routing, React Bootstrap for interface components, and Pigeon Maps for property location markers.

The current codebase is split into pages, reusable components, API service classes, auth context, custom hooks, and formatting helpers. This keeps the implementation modular without making it difficult for a new developer to follow.

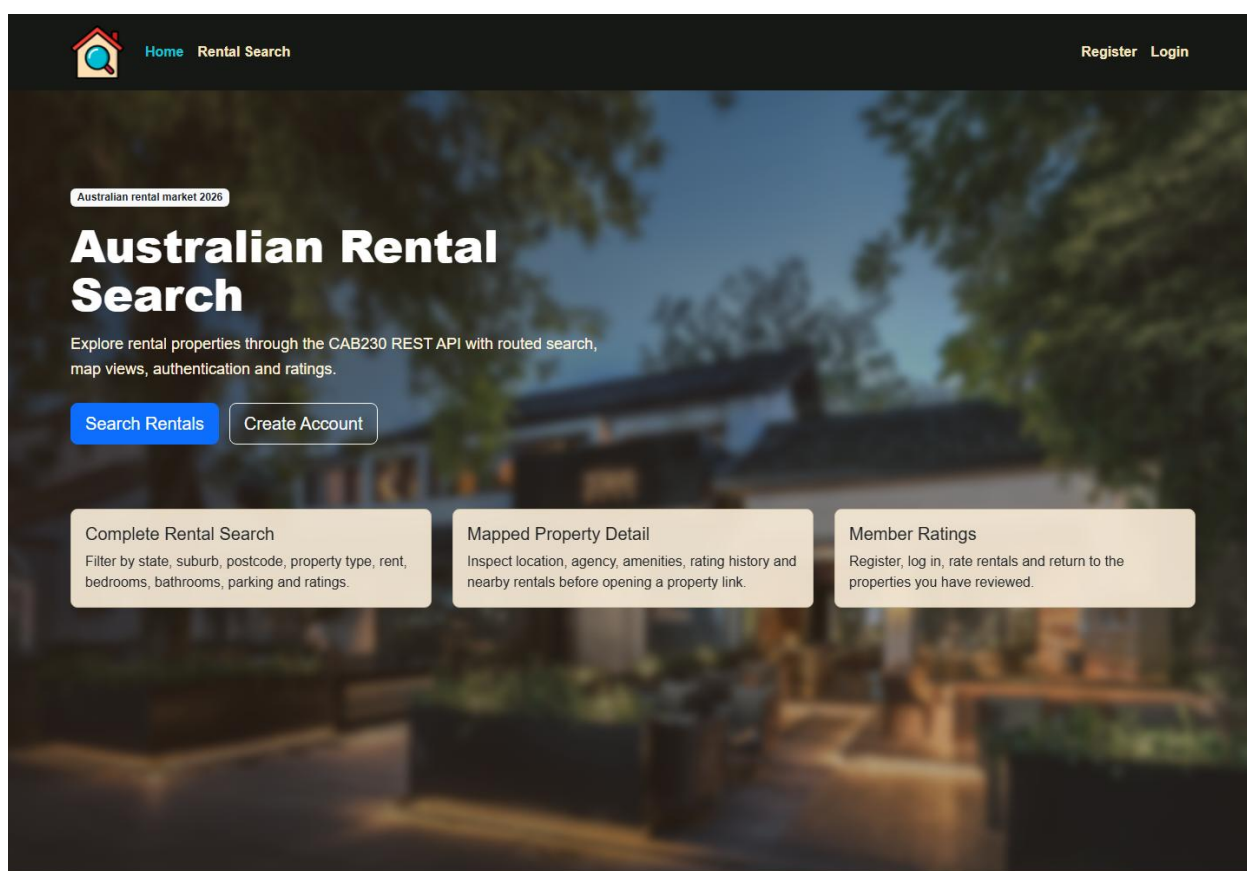


Figure 1: Home page and navigation.

[Home](#) [Rental Search](#)

Register Login

Rental Search

Search, sort and map rentals from the full CAB230 API.

6767 matches

Suburb

State

Postcode

Property Types

Search

e.g. Sydney

Any

Any

Reset

Advanced Search

Results

Page 1 of 452 - showing up to 15 rentals

Title	Rent	Type	Suburb	State	Postcode	Bed	Bath	Park	Rating
Auburn Townhouse For Lease	\$1,200	Townhouse	Auburn	NSW	2144	4	2	2	3.81 stars (390)
STUDIO ART-DECO APARTMENT	\$420	Studio	Darlinghurst	NSW	2010	3	1	2	3.56 stars (188)
A PLACE TO TRULY CALL HOME – SPACE, COMFORT & LIFESTYLE	\$580	House	Moranbah	Qld	4744	3	1	3	3.44 stars (154)
It will go quick! Call us now	\$375	Duplex/Semi-Detached	Beaudesert	Qld	4285	2	1	1	3.57 stars (121)
Stylish Waterside Living	\$630	Apartment	Thuringowa Central	Qld	4817	2	1	2	3.25 stars (99)
Convenience At Your Doorstep	\$570	Apartment	West Melbourne	Vic	3003	2	1	2	3.34 stars (80)
Elevated North-Facing Two-Bedroom Residence in the Heart of Sydney	\$1,280	Apartment	Sydney	NSW	2000	2	2	2	3.37 stars (81)
Stylish, Sunlit Living	\$590	Duplex/Semi-Detached	Holmview	Qld	4207	3	2	1	3.66 stars (68)
Fully Renovated Corner Apartment with Iconic Sydney Views	\$1,400	Apartment	Potts Point	NSW	2011	2	2	1	3.39 stars (49)
WALK TO TOWN & SCHOOL	\$550	House	Gympie	Qld	4570	3	1	2	3.46 stars (76)
Modern 4 Bedroom Townhouse Living at Greenview	\$2,100	Townhouse	Lindfield	NSW	2070	4	4	2	3.97 stars (29)
Brand New 2 Bedroom Apartment	\$1,600	Apartment	Crows Nest	NSW	2065	2	2	1	3.31 stars (76)

Figure 2: Rental search page with filters, results, pagination, and map.

Completeness and Limitations

The application implements the core public and authenticated workflows required by the assignment. Public users can search rentals and inspect detail pages. Authenticated users can submit ratings and view their rated rental history. The app consumes the lookup, search, detail, authentication, rating retrieval, and rating submission endpoints.

The application depends on the remote CAB230 API being available. If the API is unavailable or rejects a request, the user sees an error message. The main known performance limitation is the large Home background image. Automated end-to-end tests are not included; verification was performed manually and through `npm run build` and `npm run lint`.

Use of End Points

The following sections describe how the app uses each required API endpoint.

`/rentals/states`

Used by the Rental Search page to populate the State dropdown. The response is normalised in `RentalsApi` so the UI receives a simple array of states.

Relevant code: `src/api/rentalsApi.js`, `src/pages/RentalSearch.jsx`, `src/pages/rentalSearch/RentalSearchFilters.jsx`

The screenshot shows the 'Rental Search' page. At the top, there's a navigation bar with 'Home' and 'Rental Search' links, and 'Register' and 'Login' buttons. Below the navigation bar, the page title 'Rental Search' is displayed, followed by the subtitle 'Search, sort and map rentals from the full CAB230 API.' and a badge indicating '6767 matches'.

The search filters section includes:

- Suburb:** A text input field with 'e.g. Sydney' as a placeholder.
- State:** A dropdown menu currently set to 'Any'.
- Postcode:** An empty text input field.
- Property Types:** A dropdown menu currently set to 'Any'.
- Search:** A blue button to execute the search.
- Reset:** A button to clear the search filters.
- Advanced Search:** A dark button with a dropdown arrow for more search options.

Below the search filters, the 'Results' section is shown. It indicates 'Page 1 of 452 - showing up to 15 rentals'. A table lists the search results with the following columns: Title, Rent, Type, Suburb, State, Postcode, Bed, Bath, Park, and Rating.

Title	Rent	Type	Suburb	State	Postcode	Bed	Bath	Park	Rating
Auburn Townhouse For Lease	\$1,200	Townhouse	Auburn	NSW	2144	4	2	2	3.81 stars (390)
STUDIO ART-DECO APARTMENT	\$420	Studio	Darlinghurst	NSW	2010	3	1	2	3.56 stars (188)
A PLACE TO TRULY CALL HOME – SPACE, COMFORT & LIFESTYLE	\$580	House	Moranbah	Qld	4744	3	1	3	3.44 stars (154)
It will go quick! Call us now	\$375	Duplex/Semi-Detached	Beaudesert	Qld	4285	2	1	1	3.57 stars (121)
Stylish Waterside Living	\$630	Apartment	Thuringowa Central	Qld	4817	2	1	2	3.25 stars (99)
Convenience At Your Doorstep	\$570	Apartment	West Melbourne	Vic	3003	2	1	2	3.34 stars (80)
Elevated North-Facing Two-Bedroom Residence in the Heart of Sydney	\$1,280	Apartment	Sydney	NSW	2000	2	2	2	3.37 stars (81)
Stylish, Sunlit Living	\$590	Duplex/Semi-Detached	Holmview	Qld	4207	3	2	1	3.66 stars (68)
Fully Renovated Corner Apartment with Iconic Sydney Views	\$1,400	Apartment	Potts Point	NSW	2011	2	2	1	3.39 stars (49)
WALK TO TOWN & SCHOOL	\$550	House	Gympie	Qld	4570	3	1	2	3.46 stars (76)
Modern 4 Bedroom Townhouse Living at Greenview	\$2,100	Townhouse	Lindfield	NSW	2070	4	4	2	3.97 stars (29)
Brand New 2 Bedroom Apartment	\$1,600	Apartment	Crows Nest	NSW	2065	2	2	1	3.31 stars (76)

Figure 3: State filtering on the search page.

</rentals/property-types>

Used by the Rental Search page to populate the Property Types dropdown. Users can select multiple property types, and the shared API client serialises array query values as repeated query parameters.

Relevant code: `src/api/rentalsApi.js` and `src/pages/rentalSearch/PropertyTypesDropdown.jsx`

</rentals/search>

Used for the main rental search workflow. The UI supports suburb, state, postcode, property type, rent range, bedrooms, bathrooms, parking, rating range, sorting, and pagination. The UI displays 15 rentals per page and maps that onto the API page size of 10 records.

RentalSearch uses a request id guard to avoid stale responses replacing newer search results if users interact quickly.

Relevant code: `src/pages/RentalSearch.jsx` and `src/pages/rentalSearch/searchHelpers.js`


[Home](#)
[Rental Search](#)

[Register](#)
[Login](#)

6767 matches

Suburb

State

Postcode

Property Types

Search

e.g. Sydney

Any ▾

Any ▾

Reset

Advanced Search ▾

Figure 4: Search results returned from `/rentals/search`.

`/rentals/{id}`

Used on the Rental Detail page to show full property information, including rent, address, agency, amenities, description, average rating, and map location. The Rated Rentals page also uses this endpoint to turn rating records into meaningful table rows.

Relevant code: `src/pages/RentalDetail.jsx`, `src/pages/rentalDetail/*`, `src/pages/RatedRentals.jsx`


[Home](#)
[Rental Search](#)

[Register](#)
[Login](#)

[← Back to Search](#)

Auburn Townhouse For Lease

Townhouse

1/27 Louis Street, Auburn, NSW 2144

Rent
\$1,200

Bedrooms
4

Bathrooms
2

Parking
2

Description

Ideally located in a quiet setting, only a very short walk to the shops, schools and Railway station stands this large town-house and contain.

Property Features:

- 4 Good sized bedroom(all with built-in-robres) en-suite and balcony to the main bedroom.
- Sunny living/dining area with split Air conditioner.
- Ultra modern kitchen with Caesar stone bench top, gas stove and dishwasher. - Modern bathroom, internal laundry and 3 toilets.
- Double L.U Garage and storeroom.

Property Details

Agency	CAPSTONE REALTY - SYDNEY
Locality	Auburn
Amenities	Garage: 2
Average Rating	3.82 stars (391)

Your Rating

[Log in](#) to rate this rental.

Location

This map includes the selected rental and nearby rentals in the same postcode when available.

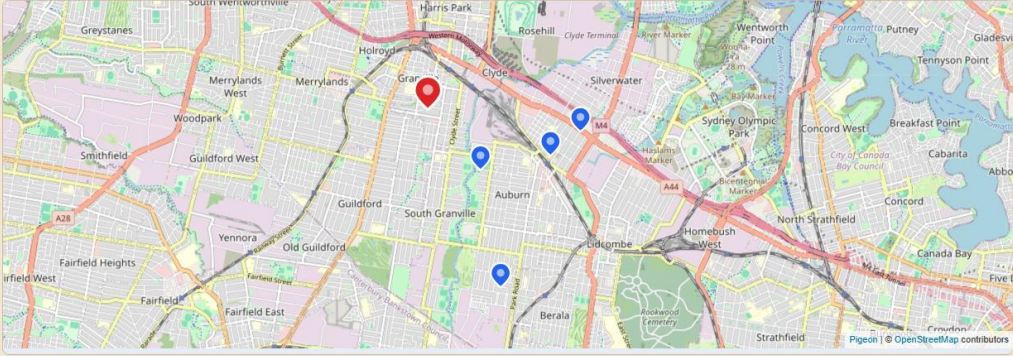


Figure 5: Rental detail page for /rentals/{id}.

/ratings

Used by the protected Rated Rentals page to retrieve the logged-in user's rating history. The page fetches rental details for each unique rentalId and caches those details to avoid repeated requests for the same property.

Relevant code: src/api/ratingsApi.js and src/pages/RatedRentals.jsx

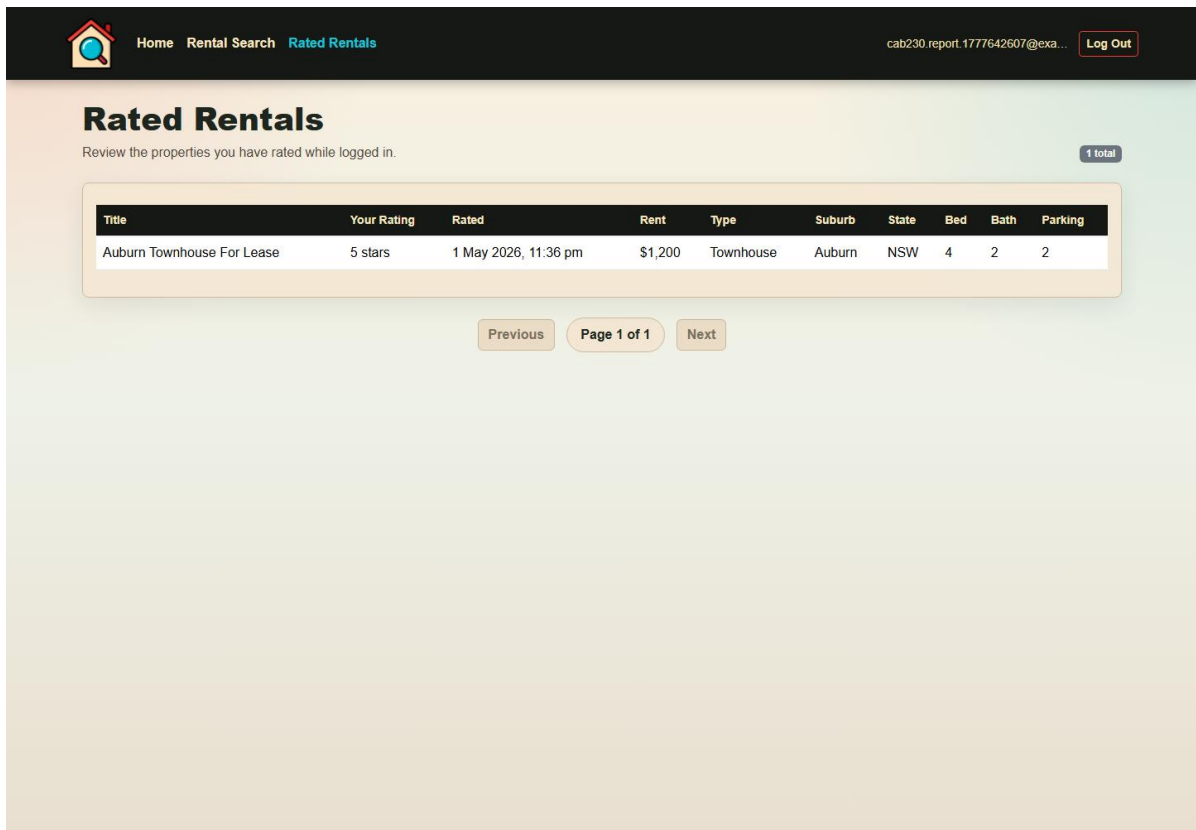


Figure 6: Rated Rentals page using /ratings.

</ratings/rentals/{id}>

Used on the Rental Detail page. GET loads the current user's existing rating for a rental; a 404 is handled gracefully when the user has not rated that property. POST submits or updates the selected rating. After submission, the detail page reloads the rental so the average rating summary is current.

Relevant code: `src/api/ratingsApi.js`, `src/pages/RentalDetail.jsx`, `src/pages/rentalDetail/RatingCard.jsx`

</user/register>

Used by the Register page to create a user account. The form validates email format, password length, and password confirmation before calling the API. Errors are displayed in a Bootstrap alert.

Relevant code: `src/api/authApi.js`, `src/pages/Register.jsx`, `src/components/FormTextField.jsx`

The screenshot shows a web application interface for a rental service. At the top, there is a dark navigation bar with a home icon, the text 'Home Rental Search', and links for 'Register' and 'Login'. The main content area has a light beige background with a subtle gradient. Centered on this background is a white rounded rectangle containing the registration form. The form is titled 'Create account' with a small 'R' logo and a subtext 'Register to save ratings against rental properties.' Below this, there are three input fields labeled 'Email', 'Password', and 'Confirm Password'. At the bottom of the form is a blue 'Create Account' button. Below the button, there is a link for 'Already registered? Log in'.

Figure 7: Register page using `/user/register`.

</user/login>

Used by the Login page to authenticate users. On success, AuthProvider stores the JWT token, email, and expiry time in local storage. Protected routes use this auth state. The API client also handles 401 responses by clearing stored auth and notifying AuthProvider that the session is no longer valid.

Relevant code: `src/api/authApi.js`, `src/context/AuthProvider.jsx`, `src/context/authSession.js`, `src/pages/Login.jsx`

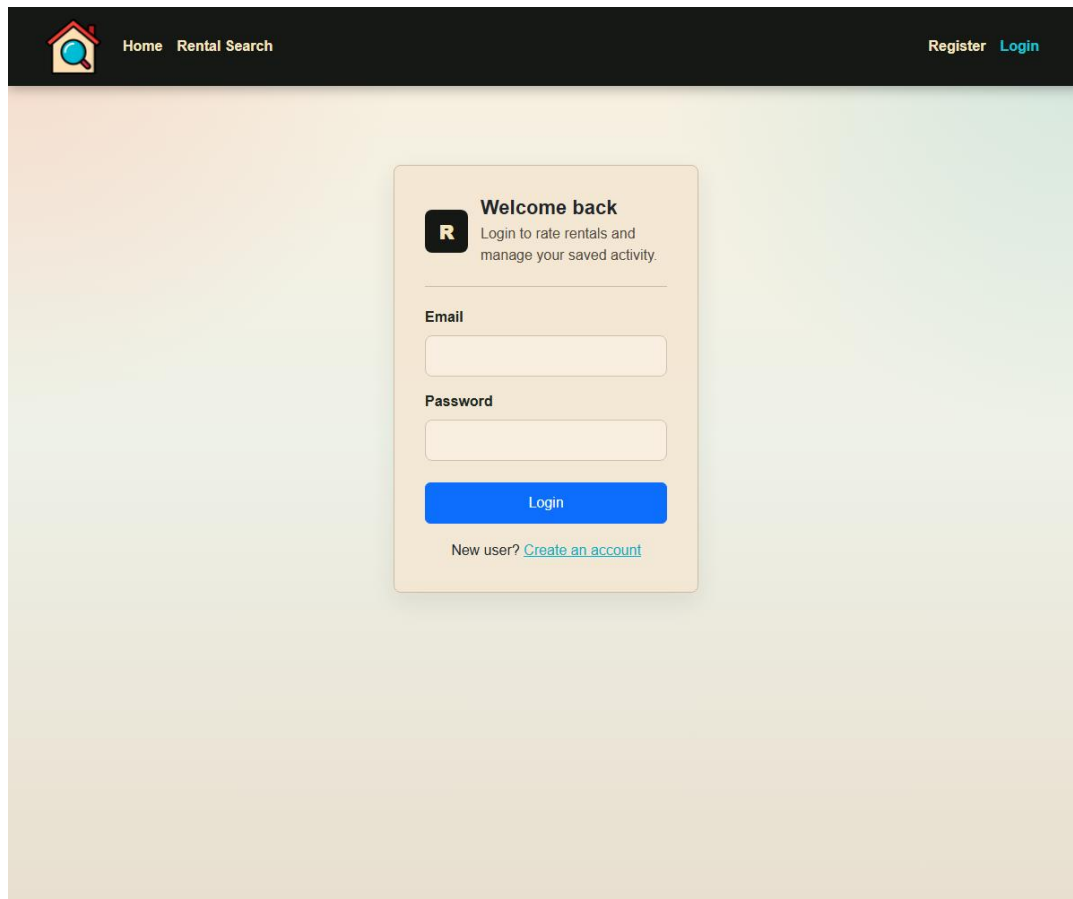


Figure 8: Login page using /user/login.

Modules Used

Module	Purpose	Documentation
react-router-dom	Routing, nested layouts, navigation, URL params, and protected routes.	https://reactrouter.com/
react-bootstrap	React UI components for navbar, forms, cards, tables, buttons, alerts, badges, and accordions.	https://react-bootstrap.github.io/
bootstrap	Base CSS and responsive layout utilities.	https://getbootstrap.com/
pigeon-maps	Interactive map and markers for rental locations.	https://github.com/mariusandra/pigeon-maps
vite	Development server and production build tool.	https://vite.dev/
eslint	Static code checking through npm run lint.	https://eslint.org/

Application Design

Navigation and Layout

The application uses a persistent top navigation bar with Home, Rental Search, Register, Login, and Rated Rentals links. Rated Rentals only appears when the user is authenticated. This keeps the public workflow simple while making authenticated functionality visible when appropriate.

The main user flow is Home -> Rental Search -> Rental Detail. Authenticated users can also use Register/Login -> Rental Detail rating -> Rated Rentals -> Rental Detail. The layout follows the task: search filters at the top, dense results in a table, pagination below results, and maps where location matters.

Usability and Quality of Design

- The interface is clean and task focused rather than decorative.
- The search controls match the data type: selects for state and sorting, checkboxes for property types, numeric inputs for numeric ranges, and buttons for actions.
- Tables are appropriate for comparing rentals because users need to scan rent, suburb, bedrooms, bathrooms, parking, and ratings together.
- Loading, empty, success, and error states are visible through reusable status components and alerts.
- The advanced search area contains many fields, but placing those controls in an accordion reduces clutter.
- The main compromise is mobile table usability. A future mobile card layout would improve smaller screens.

Accessibility

The app uses labelled form fields, text buttons, semantic headings, table headers, image alt text where images are rendered, and visible text for status messages. Information is not conveyed only by colour: selected ratings use number labels, error states use alert text, and pagination controls use text labels.

The application remains mostly understandable without CSS because the content is rendered as headings, paragraphs, forms, links, buttons, and tables. Dynamic loading is shown through a status component.

Further accessibility work would include keyboard and screen-reader testing of the map and rating controls, more detailed ARIA labels for map markers, and a text list equivalent for mapped rentals.

For inclusive design, the app should consider lower-bandwidth users and mobile access patterns, including remote and regional Aboriginal and Torres Strait Islander users. Because the app presents Australian location-based housing data, a future footer could include an Acknowledgement of Traditional Owners and a review of imagery/language for cultural accessibility.

Technical Description

Architecture

The application is a Vite React SPA. `src/main.jsx` mounts `App.jsx`. `App.jsx` defines the React Router routes, wraps the app in `AuthProvider`, and renders `Navbar` plus an `Outlet` for active pages.

API logic is centralised through `ApiClient` in `src/api/client.js`. `AuthApi`, `RentalsApi`, and `RatingsApi` are small service classes that keep fetch logic out of pages. `ApiClient` handles query strings, JSON bodies, bearer tokens, API errors, and 401 auth expiry.

Authentication lives in `src/context`. `AuthProvider` exposes `login`, `logout`, `register`, `token`, `email`, and `isAuthenticated` through `useAuth`. `ProtectedRoute` redirects unauthenticated users to `login`.

Larger pages are split into smaller components: `RentalSearch` delegates `filters/results/dropdowns/helpers`, and `RentalDetail` delegates `overview/details/rating cards`. Shared hooks handle common form field and `async-submit` state.

Source structure summary:

- `src/api` - `ApiClient` and endpoint service classes.
- `src/components` - shared UI such as `Navbar`, `RentalMap`, `StatusMessage`, `PaginationControls`, and `FormTextField`.

- src/context - auth state, stored session helpers, and auth expiry events.
- src/hooks - reusable form and async action hooks.
- src/pages - route-level screens.
- src/pages/rentalSearch - search-specific components and helpers.
- src/pages/rentalDetail - detail-specific presentation components.
- src/utils - display formatting helpers.

Test plan

Test case	Steps	Expected result	Result
Home navigation	Open / and use navbar links.	Correct pages render.	Pass
Basic rental search	Open /rentals.	Results and current page map load.	Pass
Filtered search	Apply state/property/range filters.	Results update or empty state displays.	Pass
Invalid range edge case	Set minimum greater than maximum.	Validation message prevents search.	Pass
Pagination	Use page controls.	Correct page loads without stale result replacement.	Pass
Rental detail	Click a rental row.	Detail page, description, rating panel, and map display.	Pass
Register positive	Submit valid new account details.	Account is created or API validation is shown.	Pass
Register negative	Submit mismatched passwords.	Error message displays and submit is blocked.	Pass
Login positive	Login with a valid account.	Navbar changes to authenticated state.	Pass
Protected route negative	Open /rated while logged out.	Redirects to /login.	Pass
Submit rating	Login, choose rating, submit.	Success message and refreshed rating summary.	Pass
Rated rentals	Open /rated while logged in.	Rating history table displays.	Pass
Expired auth	Use invalid or expired token.	Auth is cleared and app returns to logged-out state.	Pass
Production build	Run npm run build.	Build completes successfully.	Pass
Lint	Run npm run lint.	ESLint completes without errors.	Pass

Difficulties / Exclusions / unresolved & persistent errors

The main technical difficulty was matching API response shapes to UI needs. Lookup endpoints and search endpoints have different response structures, so lookup normalisation was placed inside RentalsApi while search pagination mapping was placed in searchHelpers.js.

Another issue was the mismatch between the API page size and the desired UI page size. The solution calculates required backend pages and slices the combined results for a 15-row UI page.

Authenticated state also needed robust handling. The app now clears local auth when authenticated API requests return 401, preventing stale logged-in UI state.

No unresolved runtime errors are known. Known limitations are the large home background image, lack of automated end-to-end tests, and limited mobile optimisation for dense tables.

Extensions

- Compress the large Home background image to improve first-load performance.
- Add a mobile card layout for search and rated rental results.
- Add saved searches or favourite rentals separate from ratings.
- Add an Acknowledgement of Traditional Owners and review cultural accessibility.
- Add Playwright end-to-end tests for search, login, rating, protected routes, and API failure scenarios.
- Move the API base URL into an environment variable.

User guide

1. Open the application and use the navbar to move between Home, Rental Search, Register, and Login.
2. Open Rental Search to load initial results.
3. Use Suburb, State, Postcode, and Property Types for basic filters.
4. Open Advanced Search to filter by rent, bedrooms, bathrooms, parking, rating, and sort options.
5. Click Search to apply filters or Reset to clear them.
6. Use pagination controls to move between result pages.
7. Click a rental row or map marker to open the Rental Detail page.
8. Review property details, description, rating summary, and map.
9. Register or log in to submit a rating.
10. Open Rated Rentals to view your rating history and click a row to return to detail.

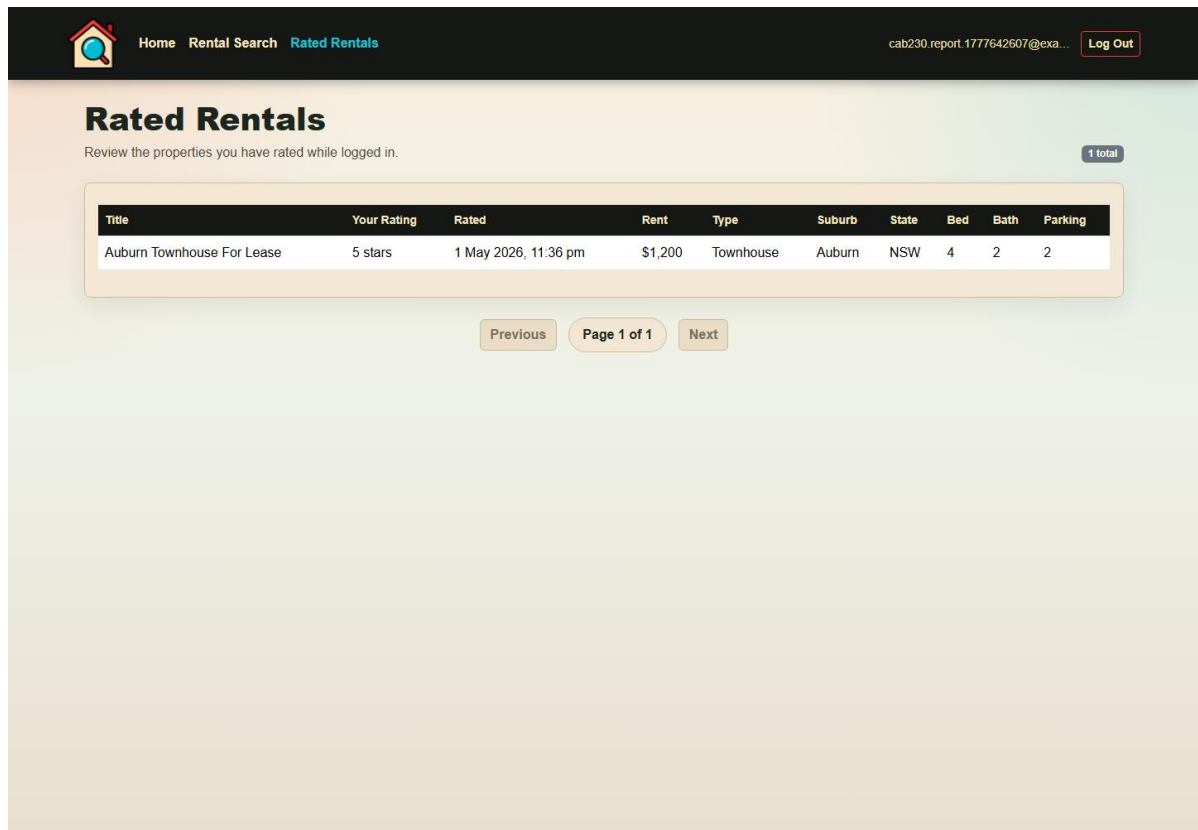


Figure 9: Authenticated rating history.

References

- React Router. <https://reactrouter.com/>
- React Bootstrap. <https://react-bootstrap.github.io/>
- Bootstrap. <https://getbootstrap.com/>
- Pigeon Maps. <https://github.com/mariusandra/pigeon-maps>
- Vite. <https://vite.dev/>
- ESLint. <https://eslint.org/>
- W3C Web Content Accessibility Guidelines checklist. <https://www.w3.org/TR/WAI-WEBCONTENT/full-checklist>

Appendices as you require them

Appendix A - Important Source Files

Area	Files
Application entry and routing	src/main.jsx, src/App.jsx
API services	src/api/client.js, src/api/authApi.js, src/api/rentalsApi.js, src/api/ratingsApi.js
Authentication	src/context/AuthProvider.jsx, src/context/authStore.js, src/context/authSession.js, src/components/ProtectedRoute.jsx
Search workflow	src/pages/RentalSearch.jsx, src/pages/rentalSearch/*
Detail workflow	src/pages/RentalDetail.jsx, src/pages/rentalDetail/*

Rated rentals workflow	src/pages/RatedRentals.jsx
Shared UI and helpers	src/components/*, src/hooks/*, src/utils/formatters.js

Appendix B - Verification Commands

The current implementation was verified with:

npm run build

npm run lint